LA(1)LR(0)

임기정

Abstract

This note fixes the LR(0) characteristic automaton, defines the one-symbol lookahead set attached to each completed LR(0) item, and derives the usual Read/Follow digraph computation. The proofs are written so that the essential claims appear as inclusions, fixed-point equations, and derivation chains.

Contents

1 The LR(0) Automaton

Definition 1 ($\mathrm{LRA}(G)$)**.** Let $G \triangleq (N, T, P, S)$ be a context-free grammar. Define:

(i)
$$G' = (N', T', P', S') \triangleq (N \uplus \{S'\},\ T \uplus \{\$\},\ P \cup \{[S' \to S\$]\},\ S').$$

(ii)
$$V \triangleq N \uplus T, \qquad V' \triangleq N' \uplus T'.$$

(iii)
$$\beta \underset{\mathbf{rm}}{\overset{}{\Longrightarrow}}_{P'} \gamma \quad \overset{\text{def}}{\Longleftrightarrow} \quad (\beta, \gamma) \in \{(\alpha A z, \alpha\omega z) \mid [A \to \omega] \in P',\ \alpha \in V'^*,\ z \in T'^*\}.$$

(iv)
$$I_{\mathrm{LR}(0)} \triangleq \{[A \to \alpha \cdot \beta] \mid [A \to \alpha\beta] \in P'\}.$$

(v) For $q \subseteq I_{\mathrm{LR}(0)}$,
$$\mathrm{Cl}(q) =_\mu q \cup \{[A \to \varepsilon \cdot \omega] \mid [A \to \omega] \in P',\ [B \to \beta \cdot A\gamma] \in \mathrm{Cl}(q)\}.$$

(vi)
$$q_0 \triangleq \mathrm{Cl}(\{[S' \to \varepsilon \cdot S\$]\}).$$

(vii) For $q \subseteq I_{\mathrm{LR}(0)}$ and $X \in V'$,
$$\mathrm{GOTO}(q, X) \triangleq \mathrm{Cl}(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in q\}).$$

(viii)
$$Q \triangleq \mathrm{PT} \setminus \{\emptyset\}, \qquad \mathrm{PT} =_\mu \{q_0\} \cup \{\mathrm{GOTO}(q, X) \mid q \in \mathrm{PT},\ X \in V'\}.$$

(ix) The path relation is the least relation satisfying
$$\varepsilon : p \to q \Longleftrightarrow p \in Q \wedge p = q, \qquad X\alpha : p \to q \Longleftrightarrow p \in Q \wedge \alpha : \mathrm{GOTO}(p, X) \to q.$$

(x)
$$\mathbf{Config} \triangleq \{(\alpha : p \to q, z) \mid \alpha \in V'^*,\ p, q \in Q,\ z \in T'^*,\ \alpha : p \to q\}.$$

(xi)
$$\delta(q, X) \triangleq \mathrm{GOTO}(q, X) \qquad (q \in Q,\ X \in V').$$

(xii)
$$\mathbf{reduce}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q\} \qquad (q \in Q,\ t \in T').$$

(xiii) Let $\vdash \subseteq \mathbf{Config} \times \mathbf{Config}$ be generated by:
$$\frac{q'' = \delta(q', t)}{(\alpha : q \to q', tz) \vdash (\alpha t : q \to q'', z)}\ \text{Shift}$$
$$\frac{\alpha : q \to p \quad \omega : p \to q' \quad [A \to \omega] \in \mathbf{reduce}(q', t) \quad q'' = \delta(p, A)}{(\alpha\omega : q \to q', tz) \vdash (\alpha A : q \to q'', tz)}\ \text{Reduce}(A \to \omega)$$

(xiv)
$$q_f \triangleq \delta(\delta(q_0, S), \$).$$

(xv) The accepted language is defined as
$$L(\mathrm{LRA}(G)) \triangleq \{z \in T^* \mid (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon)\}.$$

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, set $\mathrm{Y}(c) \in V'^*$ as

$$\mathrm{Y}(c) \triangleq \alpha u.$$

Lemma 2 (One-step yield invariant)**.** *Each parser step reverses zero or one rightmost grammar step:*

$$c \vdash c' \Longrightarrow \begin{cases} \mathrm{Y}(c') = \mathrm{Y}(c), & \text{if the step is Shift,} \\ \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mathrm{Y}(c), & \text{if the step is Reduce.} \end{cases} \tag{1}$$

Proof. Indeed,

$$\begin{aligned} \text{Shift}: \quad \mathrm{Y}(\alpha : q \to q', tz) &= \alpha t z &&= \mathrm{Y}(\alpha t : q \to q'', z), \\ \text{Reduce}: \quad \mathrm{Y}(\alpha A : q \to q'', tz) &= \alpha A t z &&\underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha\omega t z = \mathrm{Y}(\alpha\omega : q \to q', tz). \end{aligned}$$

□

Corollary 3 (Multi-step yield invariant)**.** *For $c, c' \in \mathbf{Config}$,*

$$c \vdash^* c' \Longrightarrow \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mathrm{Y}(c). \tag{2}$$

Proof. Induct on the length of $c \vdash^* c'$ and use Lemma 2 at each parser step. □

Corollary 4 (LR(0) soundness)**.**

$$L(\mathrm{LRA}(G)) \subseteq L(G).$$

Proof.

$$\begin{aligned} z \in L(\mathrm{LRA}(G)) &\Longrightarrow (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon) \\ &\overset{(2)}{\Longrightarrow} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$ \\ &\Longrightarrow S \Rightarrow^*_P z \\ &\Longrightarrow z \in L(G). \end{aligned}$$

The third implication deletes the endmarker. Write the rightmost derivation witnessing $S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$$ as

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_M = z\$.$$

Since S' occurs on no right-hand side of P', the only production mentioning S', namely $[S' \to S\$]$, is never applicable to a form derived from $S\$$; hence every step uses a production of $P \subseteq P'$. Moreover $\$ \notin V$ occurs on no right-hand side of P, and no production of P rewrites a terminal, so the rightmost symbol $\$$ of γ_0 is never touched:

$$\gamma_k = \delta_k\$, \qquad \delta_k \in V^* \quad (0 \le k \le M), \qquad \delta_0 = S, \quad \delta_M = z, \qquad \delta_k \Rightarrow_P \delta_{k+1}.$$

Stripping the common trailing $\$$ therefore yields $S = \delta_0 \Rightarrow_P \delta_1 \Rightarrow_P \cdots \Rightarrow_P \delta_M = z$, i.e. $S \Rightarrow^*_P z$. □

Fact 5 (Elimination of non-productive nonterminals)**.** *Call $A \in N'$* productive *if $A \Rightarrow^*_{P'} w$ for some $w \in T'^*$, and* non-productive *otherwise. Removing every non-productive nonterminal together with all productions that mention one preserves the generated language. Hence, whenever $L(G) \neq \emptyset$, we may assume without loss of generality that every nonterminal of G' is productive; when $L(G) = \emptyset$ the identity $L(\mathrm{LRA}(G)) = L(G) = \emptyset$ already follows from soundness (Corollary 4).*

Proof. Extend productivity to strings: $\sigma \in V'^*$ is productive if $\sigma \Rightarrow^*_{P'} x$ for some $x \in T'^*$. A terminal derives only itself and a derivation of a string is the concatenation of derivations of its symbols, so

$$Y_1 \cdots Y_m \text{ productive} \iff \forall i.\ Y_i \text{ productive} \qquad (t \in T' \text{ productive}). \tag{3}$$

FIXED-POINT CHARACTERIZATION. Let $\mathrm{Gen} \subseteq N'$ be the least fixed point

$$\mathrm{Gen} = \bigcup_{n\geq 0} \mathrm{Gen}_n, \quad \mathrm{Gen}_0 = \emptyset, \quad \mathrm{Gen}_{n+1} = \{A \in N' \mid \exists\, [A \to X_1 \cdots X_k] \in P'.\ \forall i.\ X_i \in T' \cup \mathrm{Gen}_n\}$$

(with $k = 0$, i.e. $[A \to \varepsilon] \in P'$, permitted). Then A is productive iff $A \in \mathrm{Gen}$:

$$\begin{aligned} A \in \mathrm{Gen}_{n+1} &\Longrightarrow [A \to X_1 \cdots X_k] \in P' \text{ with } X_i \in T' \cup \mathrm{Gen}_n \\ &\overset{\mathrm{IH}_n}{\Longrightarrow} A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^*_{P'} x_1 \cdots x_k \in T'^*; \\ A \Rightarrow^{\ell}_{P'} w \in T'^* \ (\ell \geq 1) &\Longrightarrow A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^{\ell-1}_{P'} w,\ X_i \Rightarrow^{\ell_i}_{P'} w_i,\ \ell_i < \ell \\ &\overset{\mathrm{IH}_\ell}{\Longrightarrow} X_i \in T' \cup \mathrm{Gen} \Longrightarrow A \in \mathrm{Gen}. \end{aligned}$$

Thus the non-productive nonterminals are exactly $N' \setminus \mathrm{Gen}$, a least fixed point.

LANGUAGE PRESERVATION. Let $\widehat{G}$ delete $N' \setminus \mathrm{Gen}$ and every production mentioning it; since terminals are productive, its productions are

$$\widehat{P} \triangleq \{[A \to X_1 \cdots X_k] \in P' \mid A, X_1, \ldots, X_k \text{ all productive}\} \subseteq P'.$$

As $\widehat{P} \subseteq P'$, $S' \Rightarrow^*_{\widehat{P}} z$ implies $S' \Rightarrow^*_{P'} z$. For the converse, induct on the length ℓ of $\sigma \Rightarrow^{\ell}_{P'} x \in T'^*$:

$$\sigma \Rightarrow^{\ell}_{P'} x \in T'^* \implies \sigma \text{ productive} \ \wedge\ \sigma \Rightarrow^*_{\widehat{P}} x.$$

For $\ell = 0$, $\sigma = x \in T'^*$. For $\ell \geq 1$, split off the first step $\sigma = \sigma_1 C \sigma_2$, $[C \to \gamma] \in P'$, $\sigma_1, \sigma_2 \in V'^*$:

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{P'} \sigma_1 \gamma \sigma_2 \Rightarrow^{\ell-1}_{P'} x \overset{\mathrm{IH}}{\Longrightarrow} \sigma_1 \gamma \sigma_2 \text{ productive} \ \wedge\ \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

By (3), γ is productive, hence so is C (via $[C \to \gamma]$), so $[C \to \gamma] \in \widehat{P}$ and

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{\widehat{P}} \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

Taking $\sigma = S'$ gives $L(G') \subseteq L(\widehat{G})$, so $L(\widehat{G}) = L(G')$ and every nonterminal of $\widehat{G}$ is productive.

When $L(G) \neq \emptyset$, S is productive, hence S' is productive ($[S' \to S\$]$) and $\widehat{G}$ keeps the start symbol with the same language; replacing G' by $\widehat{G}$ loses no generality. When $L(G) = \emptyset$, $S \notin \mathrm{Gen}$ and Corollary 4 gives $L(\mathrm{LRA}(G)) \subseteq L(G) = \emptyset$ directly. □

The path relation factors through the state reached after any prefix: induction on $|\alpha|$ using Definition 1(ix) gives, for all $\alpha, \omega \in V'^*$ and $r \in Q$,

$$\alpha\omega : r \to q \iff \exists!\, p \in Q.\ \alpha : r \to p \ \wedge\ \omega : p \to q, \tag{4}$$

the intermediate state being the unique state obtained by following α from r, since each $\delta(\cdot, X) = \mathrm{GOTO}(\cdot, X)$ is single-valued.

Lemma 6 (LR(0) path invariants). *For $\eta \in V'^*$, the characteristic automaton satisfies*

$$(\exists q \in Q)\ \eta : q_0 \to q \iff \eta \text{ is a viable prefix of } G', \tag{5}$$

$$\left.\begin{array}{c} \eta = \eta'\omega, \quad \eta' : q_0 \to p, \quad \omega : p \to q, \\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \eta'\omega y, \quad y = tz \in T'^* \end{array}\right\} \Longrightarrow [A \to \omega \cdot \varepsilon] \in q \ \wedge\ [A \to \omega] \in \mathbf{reduce}(q, t). \tag{6}$$

Proof. By Fact 5 we may assume every nonterminal of G' is productive: $C \Rightarrow^*_{P'} x$ for some $x \in T'^*$. Write $\sqsubseteq$ for the prefix order on V'^*. Call an item *valid* for $\eta \in V'^*$ when

$$[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta) \overset{\mathrm{def}}{\iff} \exists\, \delta \in V'^*, w \in T'^*.\ \eta = \delta\alpha \ \wedge\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta A w \tag{7}$$

(then $[A \to \alpha\beta] \in P'$ and $w \in T'^*$ make $\delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$ a rightmost step). Validity captures viability:

$$\mathsf{Valid}(\eta) \neq \emptyset \iff \eta \text{ is a viable prefix of } G', \tag{8}$$

since $[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta)$ yields $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$ with $\eta = \delta\alpha \sqsubseteq \delta\alpha\beta$, while a viable prefix $\eta \sqsubseteq \delta\alpha\beta$ (from $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$) factors as $\eta = \delta\alpha\beta_1$, $\beta = \beta_1\beta_2$, giving $[A \to \alpha\beta_1 \cdot \beta_2] \in \mathsf{Valid}(\eta)$.

Closure computes the valid items at a fixed η. If $q \subseteq \mathsf{Valid}(\eta)$ contains every valid item with nonempty left part (and the seed $[S' \to \varepsilon \cdot S\$]$ when $\eta = \varepsilon$), then $\mathrm{Cl}(q) = \mathsf{Valid}(\eta)$. For $\subseteq$, the closure rule adds $[A \to \varepsilon \cdot \omega]$ only from some $[B \to \beta_1 \cdot A\beta_2] \in \mathrm{Cl}(q) \subseteq \mathsf{Valid}(\eta)$, and productivity $\beta_2 \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} x \in T'^*$ gives

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta B w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\beta_1 A\beta_2 w \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta\beta_1 A x w, \qquad \eta = \delta\beta_1,\ \ xw \in T'^* \implies [A \to \varepsilon\cdot\omega] \in \mathsf{Valid}(\eta).$$

For $\supseteq$, take $[A \to \varepsilon \cdot \omega] \in \mathsf{Valid}(\eta)$ with a witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta A w$ and induct on its length. Length 0 forces $\eta A w = S'$, i.e. $\eta = \varepsilon$, $A = S'$, the seed. Otherwise the step first introducing this occurrence of A reads

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C \rho \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mu\sigma_1 A\sigma_2\rho, \qquad [C \to \sigma_1 A\sigma_2] \in P', \quad \eta = \mu\sigma_1, \quad \rho \in T'^*,$$

so $[C \to \sigma_1 \cdot A\sigma_2] \in \mathsf{Valid}(\eta)$ by the shorter witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C\rho$. This item lies in q (if $\sigma_1 \neq \varepsilon$) or in $\mathrm{Cl}(q)$ (induction hypothesis, if $\sigma_1 = \varepsilon$); its dot precedes A, so the closure rule appends $[A \to \varepsilon \cdot \omega] \in \mathrm{Cl}(q)$.

GOTO shifts the prefix. By (7), for every $X \in V'$,

$$[A \to \alpha \cdot X\beta] \in \mathsf{Valid}(\eta) \iff [A \to \alpha X \cdot \beta] \in \mathsf{Valid}(\eta X) \qquad (\text{same } \delta, w;\ \eta = \delta\alpha \iff \eta X = \delta\alpha X),$$

and every nonempty-left-part valid item for ηX ends in X, hence has this form. These items are the GOTO-kernel; closing it gives

$$q = \mathsf{Valid}(\eta) \implies \mathrm{GOTO}(q, X) = \mathsf{Valid}(\eta X). \tag{9}$$

Induction on $|\eta|$. No valid item for ε has a nonempty left part, so $q_0 = \mathrm{Cl}(\{[S' \to \varepsilon \cdot S\$]\}) = \mathsf{Valid}(\varepsilon)$. Inductively, $\eta X : q_0 \to q'$ means $\eta : q_0 \to q$ with $q = \mathsf{Valid}(\eta)$ and $q' = \delta(q, X) = \mathrm{GOTO}(q, X) \overset{(9)}{=} \mathsf{Valid}(\eta X)$. Hence

$$\eta : q_0 \to q \implies q = \mathsf{Valid}(\eta). \tag{10}$$

Because every state on a path lies in $Q = \mathrm{PT} \setminus \{\emptyset\}$, and prefixes of a viable prefix are viable (so a run never blocks),

$$(\exists q \in Q)\, \eta : q_0 \to q \overset{(10)}{\iff} \mathsf{Valid}(\eta) \neq \emptyset \overset{(8)}{\iff} \eta \text{ is a viable prefix of } G',$$

which is (5).

Handle invariant. With $\eta = \eta'\omega$, $\eta' : q_0 \to p$, $\omega : p \to q$, and $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \eta'\omega y$, $y = tz \in T'^*$: (4) gives $\eta'\omega : q_0 \to q$, so $q = \mathsf{Valid}(\eta'\omega)$ by (10). Taking $\delta = \eta'$, $w = y$ in (7),

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta' A y,\ \ y \in T'^* \implies [A \to \omega \cdot \varepsilon] \in \mathsf{Valid}(\eta'\omega) = q \overset{\text{Def. 1(xii)}}{\Longrightarrow} [A \to \omega] \in \mathbf{reduce}(q, t),$$

which is (6). □

Lemma 7 (LR(0) completeness)**.**

$$L(G) \subseteq L(\mathrm{LRA}(G)).$$

Proof. Let $z \in L(G)$, so $S \Rightarrow_P^* z$. Choose a rightmost derivation in G':

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_N = z\$. \tag{11}$$

Write its k-th step as

$$\gamma_k = \alpha_k A_k y_k \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha_k \omega_k y_k = \gamma_{k+1}, \qquad [A_k \to \omega_k] \in P', \quad y_k = t_k z_k \in T'^*. \tag{12}$$

By (6), for each k there exist $p_k, q_k \in Q$ such that

$$\alpha_k : q_0 \to p_k, \qquad \omega_k : p_k \to q_k, \qquad [A_k \to \omega_k \cdot \varepsilon] \in q_k, \qquad [A_k \to \omega_k] \in \mathbf{reduce}(q_k, t_k). \tag{13}$$

Reading (11) from right to left turns each step into one inverse reduction. By (13) and (4), $\alpha_k\omega_k : q_0 \to q_k$, so the Reduce$(A_k \to \omega_k)$ rule applies:

$$(\alpha_k\omega_k : q_0 \to q_k, y_k) \vdash (\alpha_k A_k : q_0 \to \delta(p_k, A_k), y_k) \qquad (k = N-1, \ldots, 0). \tag{14}$$

These reductions are interleaved with Shift blocks. Since $y_k \in T'^*$, the symbol A_k exposed at step k is the rightmost nonterminal of $\gamma_k = \alpha_k A_k y_k$; using the previous step $\gamma_k = \alpha_{k-1}\omega_{k-1}y_{k-1}$ with $y_{k-1} \in T'^*$ and cancelling the common terminal suffix,

$$\alpha_k A_k\, u_k = \alpha_{k-1}\omega_{k-1}, \qquad y_k = u_k\, y_{k-1}, \qquad u_k \in T'^* \qquad (1 \le k \le N-1). \tag{15}$$

By (4) the path $\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1}$ factors through $\delta(p_k, A_k)$ after the prefix $\alpha_k A_k$, so the $|u_k|$ symbols of u_k are shifted one by one:

$$(\alpha_k A_k : q_0 \to \delta(p_k, A_k),\ u_k y_{k-1}) \ \vdash^{|u_k|} \ (\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1},\ y_{k-1}).$$

At the bottom ($k = N-1$), the run starts by shifting all of $\alpha_{N-1}\omega_{N-1}$ out of $z\$ = \gamma_N$:

$$(\varepsilon : q_0 \to q_0, z\$) \ \vdash^{|\alpha_{N-1}\omega_{N-1}|} \ (\alpha_{N-1}\omega_{N-1} : q_0 \to q_{N-1}, y_{N-1}).$$

At the top ($k = 0$) we have $\gamma_0 = S\$$, whence $\alpha_0 = \varepsilon$, $A_0 = S$, $y_0 = \$$; (14) leaves the stack S reading \$, and one final Shift of \$ gives

$$(S : q_0 \to \delta(q_0, S), \$) \ \vdash \ (S\$: q_0 \to q_f, \varepsilon), \qquad q_f = \delta(\delta(q_0, S), \$).$$

Concatenating the Shift blocks with (14) for $k = N-1, \ldots, 0$ gives

$$(\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon),$$

hence $z \in L(\mathrm{LRA}(G))$. □

Corollary 8 (Correctness of $\mathrm{LRA}(G)$)**.**

$$L(G) = L(\mathrm{LRA}(G)), \qquad L(G) \triangleq \{z \in T^* \mid S \Rightarrow_P^* z\}.$$

Proof. Corollary 4 and Lemma 7 give

$$L(\mathrm{LRA}(G)) \subseteq L(G), \qquad L(G) \subseteq L(\mathrm{LRA}(G)).$$

□

2 Read and Follow Sets

Define $\mathbf{LA} : \{(q, [A \to \omega]) \mid q \in Q,\ [A \to \omega \cdot \varepsilon] \in q\} \longrightarrow \mathcal{P}(T')$ by

$$\mathbf{LA}(q, [A \to \omega]) \triangleq \left\{ t \in T' \;\middle|\; S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z,\ \alpha\omega : q_0 \to q,\ \alpha \in V'^*,\ z \in T'^* \right\}. \tag{16}$$

The remainder of this section computes $\mathbf{LA}(q, [A \to \omega])$ from the LR(0) automaton, as the least fixed point of a Read/Follow digraph.

Let

$$D \triangleq \{(p, A) \mid p \in Q,\ A \in N',\ \delta(p, A) \neq \emptyset\}.$$

For a relation R, write

$$R\,!\,x \triangleq \{y \mid (x, y) \in R\}.$$

Define $\mathbf{Read}, \mathbf{Follow} \subseteq D \times T'$ as the least relations generated by the rules

$$\frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}\,!\,(p, A)}\ \text{DR} \qquad \frac{\delta(p, A) = r \quad C \Rightarrow^*_{P'} \varepsilon \quad (r, C) \in D}{\mathbf{Read}\,!\,(r, C) \subseteq \mathbf{Read}\,!\,(p, A)}\ \text{reads}$$

$$\frac{t \in \mathbf{Read}\,!\,(p, A)}{t \in \mathbf{Follow}\,!\,(p, A)}\ \text{Read} \qquad \frac{[B \to \beta \cdot A\gamma] \in p \quad \beta : p' \to p \quad \gamma \Rightarrow^*_{P'} \varepsilon \quad (p', B) \in D}{\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A)}\ \text{includes}$$

Define the semantic read and follow sets on D:

$$\mathsf{Read}(p, A) \triangleq \{t \in T' \mid \exists r, s, \gamma.\ r = \delta(p, A) \wedge \gamma \Rightarrow^*_{P'} \varepsilon \wedge \gamma t : r \to s\}, \tag{17}$$

$$\mathsf{Follow}(p, A) \triangleq \left\{ t \in T' \;\middle|\; \exists \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z \wedge \alpha : q_0 \to p \wedge z \in T'^* \right\}. \tag{18}$$

For $(p, A) \in D$, set

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Let

$$\Phi_R(F)(p, A) = \mathrm{DR}(p, A) \cup \bigcup_{\substack{\delta(p,A)=r \\ C \Rightarrow^*_{P'} \varepsilon \\ (r,C) \in D}} F(r, C).$$

Lemma 9 (Semantic Read fixed point)**.** *For $(p, A) \in D$,*

$$\mathbf{Read}\,!\,(p, A) = \mu\Phi_R(p, A) = \mathsf{Read}(p, A). \tag{19}$$

Proof. The two **Read**-rules say exactly that **Read**! is the least Φ_R-closed family, i.e. $\mathbf{Read}!(p, A) = \mu\Phi_R(p, A)$; since Φ_R is monotone on the finite lattice $D \to \mathcal{P}(T')$,

$$\mu\Phi_R = \bigcup_{m \geq 0} \Phi_R^m(\emptyset).$$

Write $(p, A) \rightsquigarrow_R (r, C)$ for the reads edge $r = \delta(p, A) \wedge C \Rightarrow^*_{P'} \varepsilon \wedge (r, C) \in D$. Because $\Phi_R(\emptyset)(p, A) = \mathrm{DR}(p, A)$, an induction on m unfolds the operator into reads-paths:

$$\Phi_R^{m+1}(\emptyset)(p, A) = \bigcup_{i=0}^{m} \bigcup_{(p,A) \rightsquigarrow_R^i (r,C)} \mathrm{DR}(r, C). \tag{20}$$

By (17), $t \in \mathsf{Read}(p, A)$ iff for some nullable $\gamma = C_1 \cdots C_n \in N'^*$,

$$r_0 = \delta(p, A), \qquad r_i = \delta(r_{i-1}, C_i)\ (1 \leq i \leq n), \qquad \delta(r_n, t) \neq \emptyset,$$

If $n = 0$, this says exactly that $t \in \mathrm{DR}(p, A)$. If $n > 0$, it is an n-edge reads-path ending at a DR-seed holding t:

$$(p, A) \rightsquigarrow_R (r_0, C_1) \rightsquigarrow_R \cdots \rightsquigarrow_R (r_{n-1}, C_n), \qquad t \in \mathrm{DR}(r_{n-1}, C_n) \quad (\delta(\delta(r_{n-1}, C_n), t) = \delta(r_n, t) \neq \emptyset).$$

In both cases (20) puts t in $\mu\Phi_R(p, A)$, and every member of the union (20) reads back as such a nullable δ-chain. Hence (19). □

Let

$$\Phi_F(F)(p, A) = \mathsf{Read}(p, A) \cup \bigcup_{\substack{[B \to \beta \cdot A\gamma] \in p \\ \beta : p' \to p \\ \gamma \Rightarrow^*_{P'} \varepsilon \\ (p', B) \in D}} F(p', B).$$

Lemma 10 (Semantic Follow fixed point)**.** *For $(p, A) \in D$,*

$$\mathbf{Follow}\,!\,(p, A) = \mu\Phi_F(p, A) = \mathsf{Follow}(p, A). \tag{21}$$

Proof. By Lemma 9, the two **Follow**-rules define the least fixed point of Φ_F. First Follow is closed under Φ_F. For the Read-seed, fix $t \in \mathsf{Read}(p, A)$. As $p \in Q$ is reachable from q_0 by construction of PT, pick α with $\alpha : q_0 \to p$; by (17) there is a nullable γ with $\gamma t : \delta(p, A) \to s$, so (4) yields $\alpha A \gamma t : q_0 \to s$, and (5) makes $\alpha A \gamma t$ a viable prefix, i.e. $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z$ for some $z \in T'^*$. Hence

$$\begin{aligned} t \in \mathsf{Read}(p, A) &\Longrightarrow \exists \alpha, \gamma, z.\ \alpha : q_0 \to p \wedge \gamma \Rightarrow^*_{P'} \varepsilon \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z \\ &\Longrightarrow S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z \qquad (\gamma \Rightarrow^*_{P'} \varepsilon \text{ rewritten rightmost}) \\ &\Longrightarrow t \in \mathsf{Follow}(p, A). \end{aligned}$$

For an includes edge, since $t \in \mathsf{Follow}(p', B), [B \to \beta \cdot A\gamma] \in p, \beta : p' \to p,$ and $\gamma \Rightarrow^*_{P'} \varepsilon$,

$$\begin{aligned} S' &\underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z \\ &\underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t z \\ &\underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' \beta A t z, \end{aligned}$$

with $\alpha'\beta : q_0 \to p$; hence

$$\mathsf{Follow}(p', B) \subseteq \mathsf{Follow}(p, A).$$

Conversely, let $t \in \mathsf{Follow}(p, A)$, witnessed by

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha : q_0 \to p.$$

The production $B \to \beta A \gamma$ that created this occurrence of A gives a factorization

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t' z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha = \alpha'\beta, \qquad t' z' \in T'^*,$$

in which the last segment rewrites only the suffix, $\gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} t z$, leaving this A untouched. Splitting on whether γ is nullable makes exactly one alternative hold:

$$\begin{cases} \gamma = \gamma_0 X \gamma_1, \quad \gamma_0 \Rightarrow^*_{P'} \varepsilon, \quad X \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} t\chi, & t \in \mathsf{Read}(p, A), \\ \gamma \Rightarrow^*_{P'} \varepsilon, \quad t = t', & t \in \mathsf{Follow}(p', B) \text{ and } (p, A) \rightsquigarrow_{\text{includes}} (p', B). \end{cases}$$

In the first case γ_0 is read off $\delta(p, A)$ by a nullable δ-chain and $t \in \mathrm{FIRST}(X)$ gives $\delta(\cdot, t) \neq \emptyset$, so $\gamma_0 t : \delta(p, A) \to s$ realizes (17). In the second, $\gamma \Rightarrow^*_{P'} \varepsilon$ forces $t = t'$, and

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z, \quad \alpha' : q_0 \to p' \ ((4) \text{ on } \alpha = \alpha'\beta) \implies t \in \mathsf{Follow}(p', B),$$

the includes edge holding by $[B \to \beta \cdot A\gamma] \in p$, $\beta : p' \to p$, $\gamma \Rightarrow^*_{P'} \varepsilon$. Its witnessing derivation $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z$ is strictly shorter, so induction on the length gives

$$\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A).$$

Together with closure, this proves (21). □

Remark (toward a Coq formalization of the converse)**.** Every part of Lemma 10 except the converse inclusion transcribes to a proof assistant essentially verbatim: the closure direction exhibits explicit derivations and paths, so its existential witnesses are exactly the objects just constructed. The converse is the single step that does not, and it carries most of the formalization cost of this section.

The obstruction is the phrase "the production $B \to \beta A\gamma$ that created this occurrence of A", together with the factorization

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t' z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z$$

that must leave *this* A untouched. On paper the occurrence is read off the displayed string; once formalized, a right-sentential form is a bare list of symbols in which A may occur many times, and $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ records nothing about which occurrence is meant. The tracked occurrence must be promoted to first-class data, for which two encodings suffice.

1. *Marked derivations.* Refine $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ to a relation `marked_derives` $\lambda\ A\ \rho$ on a sentential form presented as $\lambda\, A\, \rho$ around a distinguished A, with one constructor per rewrite position: strictly inside the left context λ, strictly inside the right context ρ, or the introducing step $B \to \beta A\gamma$ that first plants the mark. The converse becomes structural induction on this relation; the right-context constructors yield the Read case, and the introducing constructor yields the includes edge together with the strictly shorter derivation that the induction consumes.

2. *Zipper sentential forms.* Represent the right-sentential form as a zipper (λ, A, ρ) — a left list, the focused symbol, a right list — and case-analyse each $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ step by where it fires relative to the focus: strictly left, strictly right, or at the focused symbol. Then "leaves this A untouched" is precisely the side condition that the step fires off the focus, and the same trichotomy reappears as the three rewrite positions.

Under either encoding the informal split on whether γ is nullable (Read seed versus includes edge) becomes a decidable case analysis on the focused step, and the well-founded induction runs on the length of the marked or zippered derivation rather than on an unmarked $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \cdots$. By contrast Lemma 9, Corollary 11, and Theorem 12 are first-order least-fixed-point manipulations over finite sets and need no such occurrence tracking.

Corollary 11 (Lookahead by Follow)**.** *Whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) = \{t \in T' \mid \exists p \in Q.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A)\}. \tag{22}$$

Proof. For $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,

$$\begin{aligned} t \in \mathbf{LA}(q, [A \to \omega]) &\iff \exists \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z \wedge \alpha\omega : q_0 \to q \\ &\overset{(4)}{\iff} \exists p, \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z \wedge \alpha : q_0 \to p \wedge \omega : p \to q \\ &\overset{(18)}{\iff} \exists p.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathsf{Follow}(p, A) \\ &\overset{(21)}{\iff} \exists p.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A). \end{aligned}$$

The side condition $\delta(p, A) \neq \emptyset$ is automatic for $A \neq S'$: tracing $[A \to \omega \cdot \varepsilon] \in q$ backward along $\omega : p \to q$ gives $[A \to \varepsilon \cdot \omega] \in p$, which is introduced by an item with the dot before A. For $A = S'$, both sides are empty because S' occurs on no right-hand side and the parser does not reduce $[S' \to S\$]$. This is (22). □

Theorem 12 (Digraph computation of lookahead)**.** *The least relations* $\mathbf{Read}, \mathbf{Follow}$ *generated above compute every lookahead set by the formula* (22)*.*

Proof. The Read and Follow fixed-point claims are Lemmas 9 and 10; applying them to the semantic definition of $\mathbf{LA}$ gives Corollary 11, which is the displayed formula of the theorem. □

3 Lookahead Restriction

Recall the lookahead set $\mathbf{LA}$ of (16). Replace the LR(0) reduction function by

$$\mathbf{reduce}_{\mathbf{LA}}(q,t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q,\ t \in \mathbf{LA}(q,[A \to \omega])\}. \tag{23}$$

Let $\vdash_{\mathbf{LA}}$ be the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with $\mathbf{reduce}_{\mathbf{LA}}$, and let $L(\vdash_{\mathbf{LA}})$ be its accepted language.

Lemma 13 (Guarded soundness)**.**

$$L(\vdash_{\mathbf{LA}}) \subseteq L(G).$$

Proof. From (23),

$$\mathbf{reduce}_{\mathbf{LA}}(q,t) \subseteq \mathbf{reduce}(q,t) \quad \Longrightarrow \quad \vdash_{\mathbf{LA}} \subseteq \vdash \quad \Longrightarrow \quad \vdash_{\mathbf{LA}}{}^{*} \subseteq \vdash^{*}.$$

Therefore

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\mathrm{LRA}(G)) = L(G) \qquad \text{by Corollary 8.} \tag{24}$$

□

Lemma 14 (Guarded completeness)**.**

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

Proof. Let $z \in L(G)$. In the accepting computation constructed in Lemma 7, every Reduce step has the form

$$(\alpha\omega : q_0 \to q, tz') \vdash (\alpha A : q_0 \to \delta(p,A), tz'),$$

where

$$S' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^{*}_{P'} \alpha A t z', \qquad \alpha\omega : q_0 \to q, \qquad [A \to \omega \cdot \varepsilon] \in q.$$

By (16),

$$t \in \mathbf{LA}(q,[A \to \omega]) \quad \Longleftrightarrow \quad [A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q,t).$$

Thus every Reduce step used in that accepting LR(0) computation is also legal for $\vdash_{\mathbf{LA}}$; Shift steps are unchanged. Hence

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

□

Corollary 15 (Language preservation)**.**

$$L(\vdash_{\mathbf{LA}}) = L(G).$$

Proof. Combine Lemmas 13 and 14. □

Lemma 16 (Conflict-free table condition)**.** *If the resulting table has no shift/reduce or reduce/reduce conflict, its action is single-valued under one-symbol lookahead, hence it is an* LALR(1) *table.*

Proof. At (q,t), the possible actions are

$$\delta(q,t) \quad \text{when this shift target is nonempty,} \qquad \mathbf{reduce}_{\mathbf{LA}}(q,t).$$

Absence of shift/reduce and reduce/reduce conflicts means that this action is single-valued under one-symbol lookahead, which is precisely the LALR(1) table condition. □

Theorem 17 (Lookahead-guarded reductions)**.** *If the guarded table is conflict-free, it is an* LALR(1) *parser accepting* $L(G)$*.*

Proof. Corollary 15 gives acceptance of $L(G)$, and Lemma 16 gives the LALR(1) table condition under the stated absence of conflicts. □

A Digraph Algorithm and Implementation Specification

This appendix gives an executable specification for computing the sets used in Theorem 12. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1 General Digraph Fixed-Point Problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\mathrm{Seed} : X \to \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \qquad \text{to mean} \qquad F(y) \subseteq F(x).$$

Thus the desired solution is the least function $F : X \to \mathcal{P}(T')$ satisfying

$$F(x) = \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \tag{25}$$

Algorithm DIGRAPH(X, E, Seed)**.**

1. Compute the strongly connected components of (X, E), for example by Tarjan's depth-first-search algorithm.
2. Build the condensation DAG $\mathcal{C}$. Its nodes are SCCs, and $C \rightsquigarrow D$ is an edge when some $x \in C$ and $y \in D$ satisfy $x \rightsquigarrow y$, with $C \neq D$.
3. Initialize, for every SCC C,
$$\mathrm{Val}(C) \leftarrow \bigcup_{x \in C} \mathrm{Seed}(x).$$
4. Process the SCCs in reverse topological order with respect to $\rightsquigarrow$, so every successor D of C has already been processed, and put
$$\mathrm{Val}(C) \leftarrow \mathrm{Val}(C) \cup \bigcup_{C \rightsquigarrow D} \mathrm{Val}(D).$$
5. Return $F(x) \triangleq \mathrm{Val}(C_x)$, where C_x is the SCC containing x.

Correctness condition. For every SCC C, after step 4,

$$\mathrm{Val}(C) = \bigcup \{\mathrm{Seed}(y) \mid \text{some } x \in C \text{ reaches } y \text{ in } (X, E)\}.$$

Therefore the returned F is exactly the least solution of (25). This is the only property of the graph algorithm needed by Theorem 12.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. SCC construction is $O(|X| + |E|)$, and set propagation is

$$O((|X| + |E|)b).$$

A simple work-list implementation that repeatedly applies

$$F(x) := F(x) \cup F(y) \qquad (x \rightsquigarrow y)$$

is also correct, but the SCC version avoids repeated propagation inside cycles.

A.2 Instance for Read

First compute nullable nonterminals by the least fixed point

$$\mathrm{Null}(A) \Longleftrightarrow \exists [A \to X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable},$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by

$$\mathrm{Null}(X_1 \cdots X_k) \Longleftrightarrow \bigwedge_{i=1}^{k} \mathrm{Null}(X_i).$$

For $x = (p, A) \in D$, define the direct-read seed

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \Longleftrightarrow \delta(p, A) = r \wedge C \in N' \wedge \mathrm{Null}(C) \wedge (r, C) \in D.$$

If $(r, C) \notin D$, then $\mathbf{Read}\,!\,(r, C) = \emptyset$, so no edge is needed. Compute

$$\mathbf{Read}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_R, \mathrm{DR})(p, A).$$

A.3 Instance for Follow

The seed for follow propagation is the already computed read set:

$$\mathrm{Seed}_F(p, A) \triangleq \mathbf{Read}\,!\,(p, A).$$

Define $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \Longleftrightarrow [B \to \beta \cdot A\gamma] \in p \wedge \beta : p' \to p \wedge \mathrm{Null}(\gamma) \wedge (p', B) \in D.$$

The orientation means

$$\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A).$$

Compute

$$\mathbf{Follow}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_I, \mathrm{Seed}_F)(p, A).$$

A.4 Lookback Relation and Lookahead Construction

For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define

$$\mathrm{LB}(q, A \to \omega) \triangleq \{(p, A) \in D \mid \omega : p \to q\}.$$

Then

$$\mathbf{LA}(q, [A \to \omega]) \triangleq \bigcup_{(p,A) \in \mathrm{LB}(q, A \to \omega)} \mathbf{Follow}\,!\,(p, A),$$

which is (22) written as an implementation step.

Implementation note. The query $\omega : p \to q$ can be implemented by following δ-edges from p along ω. The query $\beta : p' \to p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p), or by storing predecessor edges during construction of the LR(0) automaton.

A.5 Parser-Table and Conflict Specification

After **LA** has been computed, construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action
$$q \xrightarrow{t} \delta(q, t).$$
2. For each $[A \to \omega \cdot \varepsilon] \in q$ and each $t \in \mathbf{LA}(q, [A \to \omega])$, add the reduce action $A \to \omega$ under lookahead t.
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t).
4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t).

The accepting condition remains the one in Definition 1: the parser accepts when it reaches
$$(S\$: q_0 \to q_f, \varepsilon).$$
Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6 Summary Specification

The complete computation required by Theorem 12 is:

1. Build G', Q, and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D, DR, and E_R; compute **Read** by DIGRAPH(D, E_R, DR).
4. Build E_I; compute **Follow** by DIGRAPH$(D, E_I, \mathbf{Read})$.
5. Build LB and compute each $\mathbf{LA}(q, [A \to \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and **LA** are the least sets satisfying Theorem 12, and hence the lookahead-restricted parser of Theorem 17 uses exactly the LALR(1) lookahead sets determined by the LR(0) automaton.

B Fuel-Free Parser Execution

The table specification above is relational. An executable parser can be obtained without adding a fuel argument if the table construction also returns a termination certificate for reduce-only phases.

Definition 18 (Termination certificate)**.** For a conflict-free guarded table, let a run configuration be a stack together with the unread input. A termination certificate is a rank function
$$\rho : \mathbf{Config}_{\mathrm{run}} \to \mathbb{N}$$

such that every recursive parser step strictly decreases the lexicographic measure

$$\mu(c) \triangleq (|\mathrm{input}(c)|, \rho(c)) \in \mathbb{N} \times \mathbb{N}.$$

Concretely:

1. a Shift step consumes one terminal, so $|\mathrm{input}(c')| < |\mathrm{input}(c)|$;
2. a Reduce step consumes no input, but satisfies $\rho(c') < \rho(c)$;
3. Accept and Error configurations make no recursive call.

Definition 19 (Reduce-only summary graph)**.** Let

$$M \triangleq \max\{|\omega| \mid [A \to \omega] \in P'\}, \qquad m \triangleq M + 1.$$

Erase parse trees from the parser stack and retain only the control frames (states and grammar symbols). Let Π be the finite set of such control frames and let

$$\mathrm{suf}_m : \Pi^* \to \Pi^{\leq m}$$

take the last m frames. This suffix is enough for one Reduce step: reducing $A \to \omega$ pops at most M frames and consults the exposed predecessor state before pushing A.

For each lookahead $t \in T'$, define the finite reduce-only summary graph

$$R_t^{\#} \subseteq \Pi^{\leq m} \times \Pi^{\leq m}$$

by $s\, R_t^{\#}\, s'$ when there exist run configurations c, c' such that $\mathrm{input}(c) = tz$, the guarded table performs one Reduce step $c \to c'$, and

$$s = \mathrm{suf}_m(\mathrm{ctl}(c)), \qquad s' = \mathrm{suf}_m(\mathrm{ctl}(c')).$$

A summary-rank certificate is a family of functions

$$r_t : \Pi^{\leq m} \to \mathbb{N} \qquad (t \in T')$$

such that

$$s\, R_t^{\#}\, s' \quad \Longrightarrow \quad r_t(s') < r_t(s).$$

Equivalently, since $R_t^{\#}$ is finite, it is enough to check that each $R_t^{\#}$ is acyclic and put

$$r_t(s) \triangleq \max\Big\{ n \;\Big|\; s = s_0\, R_t^{\#}\, s_1\, R_t^{\#} \cdots R_t^{\#}\, s_n \Big\}.$$

Lemma 20 (Constructing the parser rank)**.** *Given a summary-rank certificate, define*

$$\rho(c) \triangleq \begin{cases} r_t(\mathrm{suf}_m(\mathrm{ctl}(c))), & \text{if } \mathrm{input}(c) = tz, \\ 0, & \text{if } \mathrm{input}(c) = \varepsilon. \end{cases}$$

Then every Reduce step $c \to c'$ satisfies

$$\rho(c') < \rho(c).$$

Thus ρ is the rank required in Definition 18.

Proof. A Reduce step does not consume input, so if $\mathrm{input}(c) = tz$, then $\mathrm{input}(c') = tz$ as well. By the definition of $R_t^{\#}$, the two control suffixes form an edge

$$\mathrm{suf}_m(\mathrm{ctl}(c))\, R_t^{\#}\, \mathrm{suf}_m(\mathrm{ctl}(c')).$$

The summary-rank certificate therefore gives

$$r_t(\mathrm{suf}_m(\mathrm{ctl}(c'))) < r_t(\mathrm{suf}_m(\mathrm{ctl}(c))),$$

which is exactly $\rho(c') < \rho(c)$. Shift steps are handled by the first component $|\mathrm{input}(c)|$ of μ, and Accept/Error configurations make no recursive call. □

The point is that this certificate is a property of the generated table, not a user-supplied timeout budget.

Lemma 21 (Accessibility of parser configurations)**.** *If a guarded table has a termination certificate, then every initial run configuration is accessible for the strict parser-step order*

$$c' \prec c \quad \Longleftrightarrow \quad \mu(c') <_{\text{lex}} \mu(c).$$

Proof. The strict order $<$ on $\mathbb{N}$ is well-founded, hence so is the lexicographic order on $\mathbb{N} \times \mathbb{N}$. By Definition 18, every recursive call of the parser moves from c to a configuration c' with $c' \prec c$. Therefore the inverse image of the well-founded lexicographic order along μ is well-founded on run configurations. Equivalently, every initial configuration has an $\mathsf{Acc}\ \prec$ proof. □

Corollary 22 (Fuel-free recursive parser)**.** *Under the same certificate assumption, the parser can be defined as a total recursive function*

$$\text{run}_{\mathsf{Acc}} : \prod_{c:\mathbf{Config}_{\text{run}}} \mathsf{Acc}\ \prec\ c \to \text{option ParseTree},$$

and the public parser has no fuel parameter:

$$\text{run\_parser}(w) \triangleq \text{run}_{\mathsf{Acc}}(c_0(w))(\textit{Lemma 21 applied to } c_0(w)).$$

Proof. Define $\text{run}_{\mathsf{Acc}}$ by structural recursion on the $\mathsf{Acc}\ \prec\ c$ proof. In Shift and Reduce cases, Definition 18 supplies the strict decrease needed to use the accessibility predecessor proof. In Accept and Error cases, return the corresponding result immediately. Lemma 21 supplies the accessibility proof for the initial configuration, hiding it from the public interface. □

Thus the public parser interface is

$$T^* \to \text{option ParseTree},$$

with no fuel argument and no timeout result.

References

[1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.

[2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.